VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Microprocessors – Microcontrollers (CO3009)

Lab 2 Report

Timer Interrupt and LED Scanning

Advisor(s): Dr. Lê Trọng Nhân
Student(s): Cao Nguyễn Hoàng Duy 2452190
Class: CC03

HO CHI MINH CITY, OCTOBER 2026

Contents

1 Overview **3**
1.1 Source code repository 3
1.2 Timer 2 configuration 3
1.3 Pin assignment 4

2 Exercise 1 – Two 7-segment LEDs **6**

3 Exercise 2 – Four 7-segment LEDs and DOT **8**

4 Exercise 3 – `update7SEG()` **10**

5 Exercise 4 – Scanning frequency of 1 Hz **12**

6 Exercise 5 – Digital clock **13**

7 Exercise 6 – Software timer **15**

8 Exercise 7 – Clock with software timer **17**

9 Exercise 8 – All processing in `main()` **19**

10 Exercise 9 – LED matrix **22**

11 Exercise 10 – Animation on the LED matrix **25**

1 Overview

1.1 Source code repository

All source code and simulation files of this lab are available at:

https://github.com/cnhoangduy-star/Lab2_Microprocessor

Folder	Content
`First_part/stm32`	STM32CubeIDE project `Lab2_part1` – Exercises 1–5 (timer interrupt does the work)
`First_part/proteus`	Proteus schematic `Lab2_part1.pdsprj` (4 seven-segment LEDs + DOT)
`Second_part/stm32`	STM32CubeIDE project `Lab2_part2` – Exercises 6–10 (software timers, LED matrix)
`Second_part/proteus`	Proteus schematic `Lab2_ex9.pdsprj` (previous circuit + 8×8 LED matrix)

The exercise being demonstrated is selected at compile time with the macro `EXERCISE` (values 1, 2, 4, 5 in part 1; 9, 10 in part 2). The compiled `.hex` files are kept in the `Debug` folders so that the Proteus projects can be simulated right after cloning. All listings in this report are copied directly from the two `main.c` files; the line numbers are the real line numbers in those files. Comments inside the code are written in Vietnamese.

1.2 Timer 2 configuration

The MCU is the STM32F103C6 running from the internal 8 MHz HSI clock. TIM2 is configured with the internal clock, `Prescaler = 7999` and `Period = 9`, and its global interrupt is enabled in the NVIC tab (Figure 1.1):

$$f_{\text{TIM2}} = \frac{8\,\text{MHz}}{(7999+1)(9+1)} = 100\,\text{Hz} \quad \Rightarrow \quad T_{\text{interrupt}} = 10\,\text{ms}.$$

`HAL_TIM_Base_Start_IT(&htim2)` is called in the `USER CODE BEGIN 2` section of `main()`, so `HAL_TIM_PeriodElapsedCallback` is invoked every 10 ms.

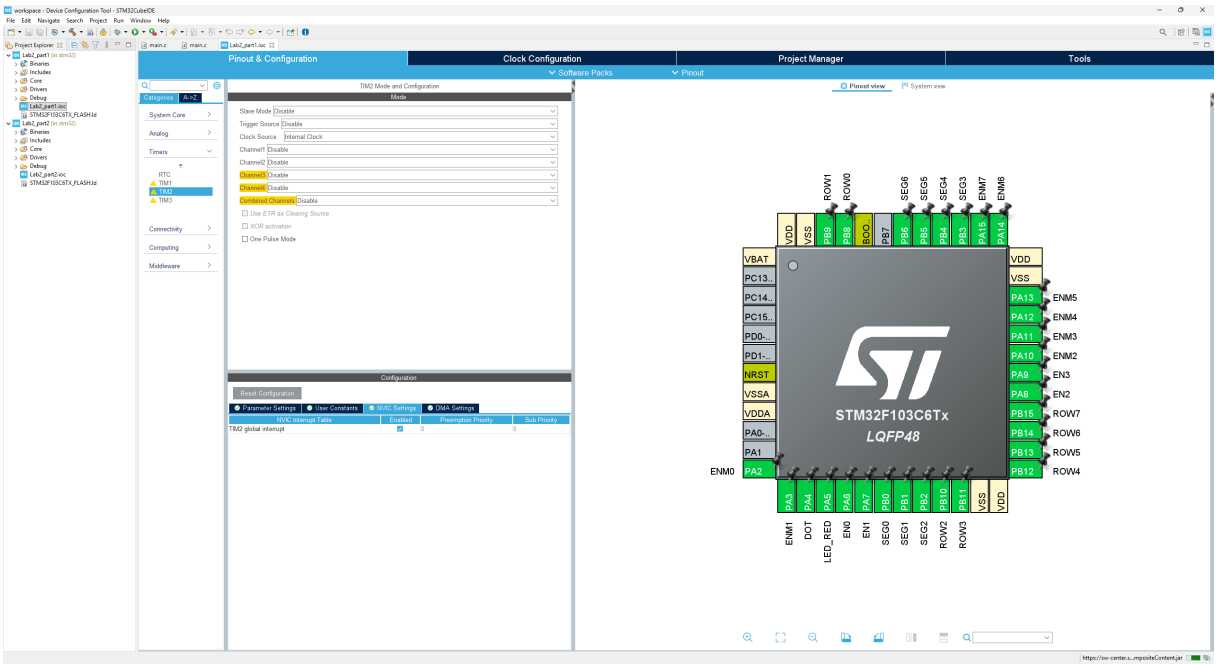


Figure 1.1: TIM2 mode (internal clock), NVIC setting and pin assignment in STM32CubeMX

1.3 Pin assignment

Signal	Pins and role
SEG0–SEG6	PB0–PB6: cathodes *a*–*g*, shared by all four common-anode 7-segment LEDs (active LOW)
EN0–EN3	PA6, PA7, PA8, PA9: base of the PNP transistors powering each 7-segment LED (active LOW)
DOT	PA4: the two LEDs in the middle of the clock
LED_RED	PA5: red LED used for the blinky demo
ROW0–ROW7	PB8–PB15: data lines of the LED matrix
ENM0–ENM7	PA2, PA3, PA10–PA15: inputs of the ULN2803 that enable one line of the LED matrix

The digit patterns are stored in `SEG_CODE` (bit 0 = segment *a*, ..., bit 6 = segment *g*). Because the displays are common-anode, a segment is lit when its pin is LOW. The same `display7SEG()` function from Lab 1 is reused (Listing 1.1).



```
78  /* Common anode: đoạn sáng khi chân SEG ở mức THẤP */
79  void display7SEG(int num) {
80    if (num < 0 || num > 9) num = 0;
81    uint8_t code = SEG_CODE[num];
82    for (int i = 0; i < 7; i++) {
83      HAL_GPIO_WritePin(SEG_PORT[i], SEG_PIN[i],
84                        ((code >> i) & 1) ? GPIO_PIN_RESET : GPIO_PIN_SET);
85    }
86  }
87
88  static void disableAll7SEG(void) {
89    for (int i = 0; i < 4; i++) {
90      HAL_GPIO_WritePin(EN_PORT[i], EN_PIN[i], GPIO_PIN_SET);  // PNP tắt khi EN ở
       mức CAO
91    }
92  }
```

Listing 1.1: `display7SEG()` and `disableAll7SEG()` *(**part1_main.c**, lines 78–92)*

2 Exercise 1 – Two 7-segment LEDs

Report 1: Schematic

Exercises 1–8 all run on the same Proteus project `Lab2_part1.pdsprj` (Figure 2.1). For Exercise 1 only the first two displays (enabled by EN0 and EN1 through PNP transistors Q1, Q2 and 1 kΩ base resistors) are used; EN2 and EN3 are kept HIGH so the other two displays stay off.

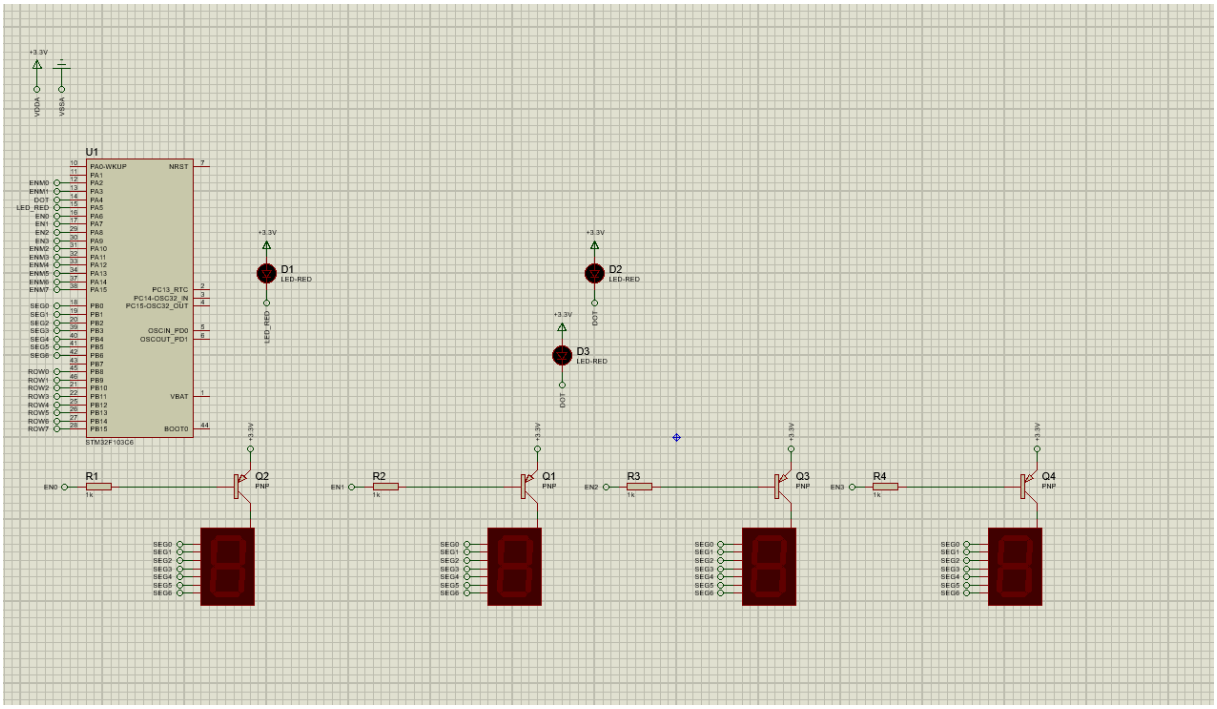


Figure 2.1: Proteus schematic of part 1: STM32F103C6, four common-anode 7-segment LEDs driven by PNP transistors, DOT LEDs (PA4) and LED_RED (PA5)

Report 2: Source code in `HAL_TIM_PeriodElapsedCallback`

`switch_counter` is reloaded with $500/10 = 50$ ticks, so every 500 ms the callback turns all displays off, writes the next digit to the segment bus and enables only one display. `led_status` remembers which display is on.

```
321 #if EXERCISE == 1 || EXERCISE == 2
322 int switch_counter = SWITCH_PERIOD_MS / TIMER_CYCLE_MS;
323 int led_status = 0;   // con 7-seg đang được bật
324 #endif
325
326 /* Được gọi mỗi 10ms */
327 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
328   if (htim->Instance != TIM2) return;
329
330 #if EXERCISE == 1
331   /* Ex1: 2 LED 7 đoạn, hiện "1" và "2", đổi LED mỗi 500ms */
332   switch_counter--;
333   if (switch_counter <= 0) {
334     switch_counter = SWITCH_PERIOD_MS / TIMER_CYCLE_MS;
335     disableAll7SEG();
336     if (led_status == 0) {
337       display7SEG(1);
338       HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, GPIO_PIN_RESET);
339       led_status = 1;
340     } else {
341       display7SEG(2);
342       HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, GPIO_PIN_RESET);
343       led_status = 0;
344     }
345   }
```

Listing 2.1: Exercise 1 – timer callback (`EXERCISE == 1`) *(`part1_main.c`, lines 321–345)*

Short question: What is the frequency of the scanning process?

Each display is on for $T_S = 500\,\text{ms}$ and there are 2 displays, so one full scan takes $2 \times 500 = 1000\,\text{ms}$. The scanning frequency is

$$f = \frac{1}{2T_S} = \frac{1}{1\,\text{s}} = 1\,\text{Hz}.$$

At this frequency the two digits are clearly seen alternating; in a real product the scanning frequency should be above about 30–50 Hz so that both digits appear to be on at the same time.



3 Exercise 2 – Four 7-segment LEDs and DOT

Report 1: Schematic

The circuit is the one in Figure 2.1: four 7-segment LEDs enabled by EN0–EN3 and the two DOT LEDs (D2, D3) connected to PA4.

Report 2: Source code in `HAL_TIM_PeriodElapsedCallback`

The four displays show `1 2 3 0` (12:30). Each display stays on for 500 ms. The DOT pin is toggled every 500 ms by a second counter, i.e. the DOT completes one ON/OFF blink cycle every second.

```
346 #elif EXERCISE == 2
347   /* Ex2: 4 LED 7 đoạn hiện "12 30", đổi LED mỗi 500ms, DOT nháy mỗi giây */
348   switch_counter--;
349   if (switch_counter <= 0) {
350     switch_counter = SWITCH_PERIOD_MS / TIMER_CYCLE_MS;
351     disableAll7SEG();
352     switch (led_status) {
353       case 0:
354         display7SEG(1);
355         HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, GPIO_PIN_RESET);
356         break;
357       case 1:
358         display7SEG(2);
359         HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, GPIO_PIN_RESET);
360         break;
361       case 2:
362         display7SEG(3);
363         HAL_GPIO_WritePin(EN2_GPIO_Port, EN2_Pin, GPIO_PIN_RESET);
364         break;
365       case 3:
366         display7SEG(0);
367         HAL_GPIO_WritePin(EN3_GPIO_Port, EN3_Pin, GPIO_PIN_RESET);
368         break;
369       default:
370         break;
371     }
372     led_status++;
373     if (led_status >= 4) led_status = 0;
374   }
375
376   dot_counter--;
377   if (dot_counter <= 0) {
378     dot_counter = DOT_PERIOD_MS / TIMER_CYCLE_MS;
379     HAL_GPIO_TogglePin(DOT_GPIO_Port, DOT_Pin);
380   }
```

Listing 3.1: Exercise 2 – timer callback (`EXERCISE == 2`) *(`part1_main.c`, lines 346–380)*

The variables used above are declared right before the callback:

```
317 /* USER CODE BEGIN 4 */
318 int scan_counter = SCAN_PERIOD_MS / TIMER_CYCLE_MS;
319 int dot_counter  = DOT_PERIOD_MS / TIMER_CYCLE_MS;
320
321 #if EXERCISE == 1 || EXERCISE == 2
322 int switch_counter = SWITCH_PERIOD_MS / TIMER_CYCLE_MS;
323 int led_status = 0;   // con 7-seg đang được bật
324 #endif
```

Listing 3.2: Counters shared by Exercises 1–5 *(part1_main.c, lines 317–324)*

Short question: What is the frequency of the scanning process?

Four displays, each on for $T_S = 500\,\text{ms}$, give a scan period of $4 \times 500 = 2000\,\text{ms}$, so

$$f = \frac{1}{4T_S} = \frac{1}{2\,\text{s}} = 0.5\,\text{Hz}.$$

4 Exercise 3 – `update7SEG()`

Report 1: Source code of `update7SEG()`

The function follows the given skeleton. It first disables all four displays (to avoid ghosting of the previous digit), then writes `led_buffer[index]` to the segment bus and enables display `index`.

```
94  /* Ex3: hiển thị led_buffer[index] lên con 7-seg thứ index */
95  void update7SEG(int index) {
96    disableAll7SEG();  // tắt hết trước để tránh bóng mờ sang con kế tiếp
97    switch (index) {
98      case 0:
99        display7SEG(led_buffer[0]);
100       HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, GPIO_PIN_RESET);
101       break;
102     case 1:
103       display7SEG(led_buffer[1]);
104       HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, GPIO_PIN_RESET);
105       break;
106     case 2:
107       display7SEG(led_buffer[2]);
108       HAL_GPIO_WritePin(EN2_GPIO_Port, EN2_Pin, GPIO_PIN_RESET);
109       break;
110     case 3:
111       display7SEG(led_buffer[3]);
112       HAL_GPIO_WritePin(EN3_GPIO_Port, EN3_Pin, GPIO_PIN_RESET);
113       break;
114     default:
115       break;
116   }
117 }
```

Listing 4.1: `update7SEG()` *(`part1_main.c`, lines 94–117)*

The buffer and index are declared as in the skeleton:

```
52  const int MAX_LED = 4;
53  int index_led = 0;
54  volatile int led_buffer[4] = {1, 2, 3, 4};
55  int hour = 15, minute = 8, second = 50;
```

Listing 4.2: Global variables for the 7-segment buffer and the clock *(`part1_main.c`, lines 52–55)*

Report 2: Source code in `HAL_TIM_PeriodElapsedCallback`

For Exercises 3, 4 and 5 the callback calls `update7SEG(index_led++)` every `SCAN_PERIOD_MS` and wraps `index_led` back to 0 after 3, so it always stays in the valid range 0–3. The DOT keeps blinking with its own counter.

```
381 #else
382   /* Ex3, Ex4, Ex5: quét 4 LED bằng update7SEG */
383   scan_counter--;
384   if (scan_counter <= 0) {
385     scan_counter = SCAN_PERIOD_MS / TIMER_CYCLE_MS;
386     update7SEG(index_led++);
387     if (index_led >= MAX_LED) index_led = 0;
388   }
389
390   dot_counter--;
391   if (dot_counter <= 0) {
392     dot_counter = DOT_PERIOD_MS / TIMER_CYCLE_MS;
393     HAL_GPIO_TogglePin(DOT_GPIO_Port, DOT_Pin);
394   }
395 #endif
396 }
```

Listing 4.3: Exercises 3–5 – timer callback (default branch) *(part1_main.c, lines 381–396)*

With the initial buffer `{1, 2, 3, 4}` the displays show 1, 2, 3, 4 in turn; changing the values in `led_buffer` changes the displayed digits without touching the scanning code, which is how the function was unit-tested.

5 Exercise 4 – Scanning frequency of 1 Hz

Report 1: Source code in `HAL_TIM_PeriodElapsedCallback`

To scan all four displays at $f = 1\,\text{Hz}$, each display must be on for $T_S = \frac{1}{4f} = 250\,\text{ms}$, i.e. $250/10 = 25$ timer interrupts. Only the constants change; the callback is the one in Listing 4.3.

```
36 #define EXERCISE        4
37 #define TIMER_CYCLE_MS  10    // TIM2: 8MHz/(7999+1)/(9+1) = 100Hz -> ngắt mỗi
   10ms
38 #define SWITCH_PERIOD_MS 500  // Ex1, Ex2: mỗi LED 7 đoạn sáng 500ms
39 #define SCAN_PERIOD_MS  250   // mỗi LED sáng 250ms -> quét đủ 4 LED mất 1s -> tần
   số quét 1Hz
40 #define DOT_PERIOD_MS   500   // DOT đảo trạng thái mỗi 500ms -> nháy 1 lần mỗi
   giây
```

Listing 5.1: Timing constants (`EXERCISE == 4`: 250 ms per display, DOT toggles every 500 ms) *(`part1_main.c`, lines 36–40)*

```
383 scan_counter--;
384 if (scan_counter <= 0) {
385   scan_counter = SCAN_PERIOD_MS / TIMER_CYCLE_MS;
386   update7SEG(index_led++);
387   if (index_led >= MAX_LED) index_led = 0;
388 }
389
390 dot_counter--;
391 if (dot_counter <= 0) {
392   dot_counter = DOT_PERIOD_MS / TIMER_CYCLE_MS;
393   HAL_GPIO_TogglePin(DOT_GPIO_Port, DOT_Pin);
394 }
```

Listing 5.2: Exercise 4 – body of the timer callback *(`part1_main.c`, lines 383–394)*

The DOT is still toggled every 500 ms, so it blinks once every second.

6 Exercise 5 – Digital clock

Report 1: Source code of `updateClockBuffer()`

The hour and minute are split into tens and units. Using `/10` for the first digit automatically produces a leading `0` for one-digit values (e.g. 8 minutes → `0`, `8`).

```
119 /* Ex5: tách giờ/phút thành 4 chữ số, tự thêm số 0 khi chỉ có 1 chữ số */
120 void updateClockBuffer(void) {
121   led_buffer[0] = hour / 10;
122   led_buffer[1] = hour % 10;
123   led_buffer[2] = minute / 10;
124   led_buffer[3] = minute % 10;
125 }
```

Listing 6.1: `updateClockBuffer()` *(`part1_main.c`, lines 119–125)*

The main loop follows the given skeleton (blocking version, still using `HAL_Delay`):

```
157   /* USER CODE BEGIN 2 */
158   disableAll7SEG();
159 #if EXERCISE == 5
160   updateClockBuffer();
161 #endif
162   HAL_TIM_Base_Start_IT(&htim2);
163   /* USER CODE END 2 */
164
165   /* Infinite loop */
166   /* USER CODE BEGIN WHILE */
167   while (1)
168   {
169     /* USER CODE END WHILE */
170
171     /* USER CODE BEGIN 3 */
172 #if EXERCISE == 5
173     second++;
174     if (second >= 60) { second = 0; minute++; }
175     if (minute >= 60) { minute = 0; hour++; }
176     if (hour >= 24)   { hour = 0; }
177     updateClockBuffer();
178     HAL_Delay(1000);
179 #endif
180   }
```

Listing 6.2: Exercise 5 – `main()` initialisation and loop (`EXERCISE == 5`) *(`part1_main.c`, lines 157–180)*

Starting from 15:08:50, the display shows `15 08` and changes to `15 09` after 10 seconds.

7 Exercise 6 – Software timer

From this exercise on, the project `Lab2_part2` is used. The single software timer of the manual (`timer0_counter`, `timer0_flag`) is generalised into arrays so that one timer can be created for every periodic task; `setTimer(id, duration)` and `timer_run()` behave exactly like `setTimer0()` and the manual's `timer_run()` for each timer. The arrays are `volatile` because they are written inside the interrupt and read in `main()`.

```
38 #define TIMER_CYCLE       10    // TIM2: 8MHz/(7999+1)/(9+1) = 100Hz -> ngắt mỗi
   10ms
39 #define NUM_TIMERS        6     // số software timer
40
41 /* Chỉ số software timer */
42 #define T_CLOCK           0     // 1000ms: đếm giây + nháy DOT (Ex7)
43 #define T_SCAN_7SEG       1     // quét 7-seg (Ex8)
44 #define T_SCAN_MATRIX     2     // quét LED matrix (Ex9)
45 #define T_SHIFT           3     // dịch chữ (Ex10)
46 #define T_LED_RED         4     // ví dụ Program 1.8 (Ex6)
47 #define T_SELFTEST        5     // chế độ dò phân cực matrix
48
49 #define SCAN_7SEG_MS      250   // 4 LED x 250ms = 1s -> quét 1Hz như Ex4
50 #define SCAN_MATRIX_MS    10    // mỗi cột 10ms -> 8 cột = 80ms/khung (đổi thành
   200 để xem từng cột sáng lần lượt)
51 #define SHIFT_MS          200   // Ex10: dịch 1 cột mỗi 200ms
```

Listing 7.1: Software timer IDs and periods *(`part2_main.c`, lines 38–51)*

```
76 /* ---------- Software timer (Ex6) ---------- */
77 volatile int timer_counter[NUM_TIMERS] = {0};
78 volatile int timer_flag[NUM_TIMERS]    = {0};
```

Listing 7.2: Software timer counters and flags *(`part2_main.c`, lines 76–78)*



```
140 /* ================ Software timer (Ex6) ================ */
141 void setTimer(int id, int duration) {
142   timer_counter[id] = duration / TIMER_CYCLE;
143   timer_flag[id] = 0;
144 }
145
146 /* Gọi trong ngắt mỗi TIMER_CYCLE ms: chỉ đếm, không xử lý gì nặng */
147 void timer_run(void) {
148   for (int i = 0; i < NUM_TIMERS; i++) {
149     if (timer_counter[i] > 0) {
150       timer_counter[i]--;
151       if (timer_counter[i] == 0) timer_flag[i] = 1;
152     }
153   }
154 }
```

Listing 7.3: `setTimer()` and `timer_run()` *(`part2_main.c`, lines 140–154)*

```
503 /* USER CODE BEGIN 4 */
504 /* Ex8: ngắt chỉ còn chạy software timer */
505 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
506   if (htim->Instance == TIM2) {
507     timer_run();
508   }
509 }
510 /* USER CODE END 4 */
```

Listing 7.4: `timer_run()` is the only work done in the timer interrupt *(`part2_main.c`, lines 503–510)*

The LED blinky example of Program 1.8 is implemented with timer `T_LED_RED`: it is started with `setTimer(T_LED_RED, 1000)` before the loop (line 307), and the loop contains:

```
361 /* Ex6: ví dụ Program 1.8 - LED đỏ PA5 */
362 if (timer_flag[T_LED_RED] == 1) {
363   HAL_GPIO_TogglePin(LED_RED_GPIO_Port, LED_RED_Pin);
364   setTimer(T_LED_RED, 2000);
365 }
```

Listing 7.5: Program 1.8 in the main loop *(`part2_main.c`, lines 361–365)*



Report 1: If line 1 (`setTimer0(1000)`) is missing, what happens and why?

The LED never toggles. The counter and the flag both start at 0. `timer_run()` only decrements the counter when `timer0_counter > 0`, and the flag is only set when the counter goes from 1 to 0. With a counter of 0 nothing ever happens, `timer0_flag` stays 0, the `if` in the loop is never true, and `setTimer0(2000)` is never reached either. The LED stays in its initial state forever.

Report 2: If line 1 is changed to `setTimer0(1)`, what happens and why?

The result is the same as in Report 1: the LED never blinks. `setTimer0()` computes `duration / TIMER_CYCLE` $= 1/10 = 0$ because of integer division, so the counter is again 0 and the flag is cleared. The timer is therefore never started. Any duration smaller than `TIMER_CYCLE` (10 ms) has this problem.

Report 3: If line 1 is changed to `setTimer0(10)`, what changes and why?

Now $10/10 = 1$, so the counter is 1. At the next timer interrupt it becomes 0 and `timer0_flag` is set. The LED toggles almost immediately (within at most 10 ms, because the first interrupt can come at any moment within the current 10 ms period), and from then on it is reloaded with `setTimer0(2000)` and toggles every 2 s. Compared to the first two cases the program works correctly; compared to `setTimer0(1000)` only the first toggle happens earlier (about 10 ms instead of 1 s). This shows that the smallest usable duration is one timer cycle, and that durations are truncated to a multiple of `TIMER_CYCLE`.

8 Exercise 7 – Clock with software timer

Report 1: Source code in the while loop of `main()`

Timer `T_CLOCK` expires every 1000 ms. When its flag is set, the timer is reloaded, the time is advanced by one second, the 7-segment buffer is updated and the DOT is toggled. `HAL_Delay(1000)` is removed, so the loop never blocks.

```
319     /* Ex7: đồng hồ + DOT chạy bằng software timer, không còn HAL_Delay */
320     if (timer_flag[T_CLOCK] == 1) {
321       setTimer(T_CLOCK, 1000);
322       second++;
323       if (second >= 60) { second = 0; minute++; }
324       if (minute >= 60) { minute = 0; hour++; }
325       if (hour >= 24)   { hour = 0; }
326 #if !MATRIX_SELFTEST
327       updateClockBuffer();
328 #endif
329       HAL_GPIO_TogglePin(DOT_GPIO_Port, DOT_Pin);
330     }
```

Listing 8.1: Exercise 7 – clock and DOT in the main loop *(part2_main.c, lines 319–330)*

The timers are started before entering the loop:

```
290   /* USER CODE BEGIN 2 */
291   disableAll7SEG();
292 #if MATRIX_SELFTEST
293   applySelftestCombo(0);
294   setTimer(T_SELFTEST, 5000);
295 #else
296   updateClockBuffer();
297 #if EXERCISE == 10
298   shiftLeftAnimation();
299 #endif
300 #endif
301   matrixAllOff();
302
303   setTimer(T_CLOCK, 1000);
304   setTimer(T_SCAN_7SEG, SCAN_7SEG_MS);
305   setTimer(T_SCAN_MATRIX, SCAN_MATRIX_MS);
306   setTimer(T_SHIFT, SHIFT_MS);
307   setTimer(T_LED_RED, 1000);              // Program 1.8 (Ex6)
308
309   HAL_TIM_Base_Start_IT(&htim2);
310   /* USER CODE END 2 */
```

Listing 8.2: Initialisation in `main()` (`USER CODE BEGIN 2`) *(part2_main.c, lines 290–310)*



9 Exercise 8 – All processing in `main()`

Report 1: Source code in `main()` and extra functions

`update7SEG()` is moved from the interrupt to the main loop and is driven by a second software timer, `T_SCAN_7SEG` (250 ms per display, i.e. 1 Hz scanning as in Exercise 4). The timer callback now only runs `timer_run()` (Listing 7.4); all other work is done in the super loop below. The extra functions used are `setTimer()` / `timer_run()` (Listing 7.3), `update7SEG()` and `updateClockBuffer()` (same code as in part 1).

```
312   /* Infinite loop */
313   /* USER CODE BEGIN WHILE */
314   while (1)
315   {
316     /* USER CODE END WHILE */
317
318     /* USER CODE BEGIN 3 */
319     /* Ex7: đồng hồ + DOT chạy bằng software timer, không còn HAL_Delay */
320     if (timer_flag[T_CLOCK] == 1) {
321       setTimer(T_CLOCK, 1000);
322       second++;
323       if (second >= 60) { second = 0; minute++; }
324       if (minute >= 60) { minute = 0; hour++; }
325       if (hour >= 24)   { hour = 0; }
326 #if !MATRIX_SELFTEST
327       updateClockBuffer();
328 #endif
329       HAL_GPIO_TogglePin(DOT_GPIO_Port, DOT_Pin);
330     }
331
332     /* Ex8: quét 7-seg chuyển từ ngắt về main */
333     if (timer_flag[T_SCAN_7SEG] == 1) {
334       setTimer(T_SCAN_7SEG, SCAN_7SEG_MS);
335       update7SEG(index_led++);
336       if (index_led >= MAX_LED) index_led = 0;
337     }
338
339     /* Ex9: quét LED matrix trong main */
340     if (timer_flag[T_SCAN_MATRIX] == 1) {
341       setTimer(T_SCAN_MATRIX, SCAN_MATRIX_MS);
342       updateLEDMatrix(index_led_matrix++);
343       if (index_led_matrix >= MAX_LED_MATRIX) index_led_matrix = 0;
344     }
345
```

Listing 9.1: Exercise 8 – super loop of `main()` (part 1/2) *(**part2_main.c**, lines 312–345)*

```
346  #if MATRIX_SELFTEST
347      if (timer_flag[T_SELFTEST] == 1) {
348        setTimer(T_SELFTEST, 5000);
349        selftest_combo = (selftest_combo + 1) % 4;
350        matrixAllOff();                        // tắt theo phân cực cũ trước khi đổi
351        applySelftestCombo(selftest_combo);
352      }
353  #elif EXERCISE == 10
354      /* Ex10: dịch chữ sang trái */
355      if (timer_flag[T_SHIFT] == 1) {
356        setTimer(T_SHIFT, SHIFT_MS);
357        shiftLeftAnimation();
358      }
359  #endif
360
361      /* Ex6: ví dụ Program 1.8 - LED đỏ PA5 */
362      if (timer_flag[T_LED_RED] == 1) {
363        HAL_GPIO_TogglePin(LED_RED_GPIO_Port, LED_RED_Pin);
364        setTimer(T_LED_RED, 2000);
365      }
366    }
367    /* USER CODE END 3 */
```

Listing 9.2: Exercise 8 – super loop of `main()` (part 2/2) *(**part2_main.c**, lines 346–367)*

Each task checks its own flag and reloads its own timer, so the tasks are independent and the interrupt handler stays very short (a loop over 6 counters), which avoids the nested-interrupt problem described in the manual.

10 Exercise 9 – LED matrix

Report 1: Schematic

A MATRIX-8X8-RED and a ULN2803 are added to the previous circuit (Figure 10.1). ROW0–ROW7 (PB8–PB15) drive the data pins of the matrix. ENM0–ENM7 drive the inputs of the ULN2803; when an input is HIGH the corresponding Darlington output sinks the matrix line COLk to ground, so exactly one line of the matrix is enabled at a time.

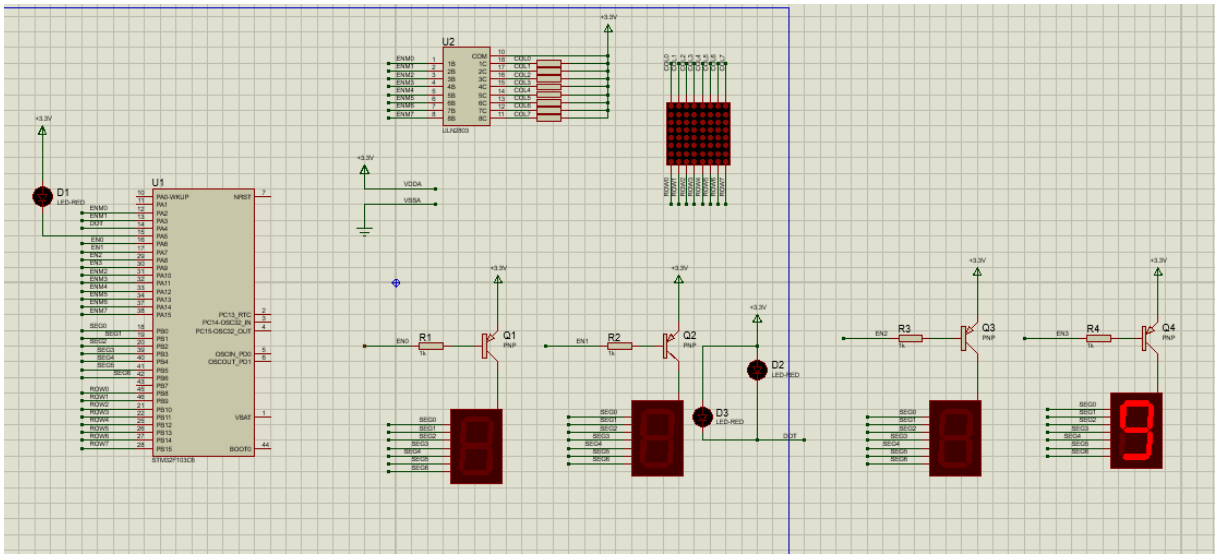


Figure 10.1: Proteus schematic of part 2: the circuit of part 1 plus ULN2803 (U2) and the 8×8 LED matrix

Report 2: `updateLEDMatrix()`

`matrix_buffer[c]` stores the 8 pixels of line `c`, bit `r` being row `r`. Instead of eight identical `case` blocks, the function uses lookup tables of the ENM and ROW pins, so `index` directly selects the line. For each call it (1) turns all lines off to avoid ghosting, (2) writes the 8 data bits to ROW0–ROW7, and (3) enables line `index`. The ON levels are macros (`MATRIX_COL_ON`, `MATRIX_ROW_ON`) so the polarity can be changed without touching the function.



```
204 /* ================= LED matrix (Ex9) ================= */
205 static void matrixAllOff(void) {
206   GPIO_PinState colOff = (matrix_col_on == GPIO_PIN_SET) ? GPIO_PIN_RESET :
    GPIO_PIN_SET;
207   for (int i = 0; i < 8; i++) {
208     HAL_GPIO_WritePin(ENM_PORT[i], ENM_PIN[i], colOff);
209   }
210 }
211
212 /* Dữ liệu cho đường quét thứ k (có hỗ trợ xoay 90 độ nếu matrix nối ngược
    hàng/cột) */
213 static uint8_t getLineData(int k) {
214 #if MATRIX_TRANSPOSE
215   uint8_t d = 0;
216   for (int j = 0; j < 8; j++) {
217     if ((matrix_buffer[j] >> k) & 1) d |= (uint8_t)(1 << j);
218   }
219   return d;
220 #else
221   return matrix_buffer[k];
222 #endif
223 }
```

Listing 10.1: Helper functions `matrixAllOff()` and `getLineData()` *(`part2_main.c`, lines 204–223)*

```
225 /* Bật duy nhất cột index, xuất dữ liệu cột đó ra 8 đường ROW */
226 void updateLEDMatrix(int index) {
227   if (index < 0 || index >= MAX_LED_MATRIX) return;
228   GPIO_PinState rowOff = (matrix_row_on == GPIO_PIN_SET) ? GPIO_PIN_RESET :
      GPIO_PIN_SET;
229
230   int line = MATRIX_LINE_REVERSE ? (7 - index) : index;  // đường ENM thật sẽ bật
231
232   matrixAllOff();                        // 1. tắt hết đường quét để không bị bóng
      mờ
233   uint8_t data = getLineData(index);
234   for (int r = 0; r < 8; r++) {          // 2. xuất dữ liệu ra 8 đường ROW
235     int pin = MATRIX_ROW_REVERSE ? (7 - r) : r;
236     HAL_GPIO_WritePin(ROW_PORT[pin], ROW_PIN[pin], ((data >> r) & 1) ?
      matrix_row_on : rowOff);
237   }
238   HAL_GPIO_WritePin(ENM_PORT[line], ENM_PIN[line], matrix_col_on);    // 3. bật
      đường quét
239 }
```

Listing 10.2: `updateLEDMatrix()` *(`part2_main.c`, lines 225–239)*

The buffer is updated to display the character “A”:

```
102 /* matrix_buffer[c] = dữ liệu cột c (trái -> phải); bit r = hàng r (bit0 = hàng
    trên cùng)
103  *
104  *   . . . # # . . .
105  *   . . # . . # . .
106  *   . # . . . . # .
107  *   . # . . . . # .
108  *   . # # # # # # .
109  *   . # . . . . # .
110  *   . # . . . . # .
111  *   . . . . . . . .
112  */
113 uint8_t matrix_buffer[8] = {0x00, 0x7C, 0x12, 0x11, 0x11, 0x12, 0x7C, 0x00};
```

Listing 10.3: `matrix_buffer` for the character “A” *(`part2_main.c`, lines 102–113)*

`updateLEDMatrix()` is invoked in `main()` by timer `T_SCAN_MATRIX` (lines 339–344 in Listing 9.1) every 10 ms, so one frame of 8 lines takes 80 ms (12.5 Hz). This low rate was chosen because Proteus cannot simulate fast scanning in real time; on real hardware the period per line would be about 2 ms ($>$ 50 Hz per frame).

11 Exercise 10 – Animation on the LED matrix

Report 1: Solution and source code

The animation shifts the character "A" to the left. A 16-column frame `anim_frame` holds the character followed by 8 blank columns. Every `SHIFT_MS` = 200 ms (timer `T_SHIFT`) the function `shiftLeftAnimation()` copies an 8-column window of this frame, starting at `anim_offset`, into `matrix_buffer`, then increases the offset modulo 16. Each step therefore moves the picture one column to the left; after 16 steps (3.2 s) the character has left the screen completely and enters again from the right. The scanning task of Exercise 9 is not changed: it simply displays whatever is in `matrix_buffer`.

```
115 /* Ex10: khung hình 16 cột = chữ "A" + 8 cột trống, cửa sổ 8 cột trượt sang trái
    */
116 static const uint8_t anim_frame[16] = {0x00, 0x7C, 0x12, 0x11, 0x11, 0x12, 0x7C,
    0x00,
117                                         0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
                                            0x00};
118 int anim_offset = 0;
```

Listing 11.1: Animation frame and offset *(part2_main.c, lines 115–118)*

```
241 /* ================= Animation (Ex10) ================= */
242 /* Cửa sổ 8 cột trượt trên khung 16 cột -> chữ "A" chạy sang trái rồi quay vòng */
243 void shiftLeftAnimation(void) {
244   for (int i = 0; i < 8; i++) {
245     matrix_buffer[i] = anim_frame[(i + anim_offset) % 16];
246   }
247   anim_offset = (anim_offset + 1) % 16;
248 }
```

Listing 11.2: `shiftLeftAnimation()` *(part2_main.c, lines 241–248)*

```
353 #elif EXERCISE == 10
354     /* Ex10: dịch chữ sang trái */
355     if (timer_flag[T_SHIFT] == 1) {
356       setTimer(T_SHIFT, SHIFT_MS);
357       shiftLeftAnimation();
358     }
359 #endif
```

Listing 11.3: Exercise 10 – shift task in the main loop (`EXERCISE == 10`) *(`part2_main.c`, lines 353–359)*

The first frame is prepared once before the loop (`shiftLeftAnimation()` at line 298 in Listing 8.2), so the matrix shows a valid picture from the first scan.